

Computer Programming: Lecture 3

Outline

- Boolean Expressions
- Functions

Boolean Expressions

- A Boolean expression is an expression that evaluates to either **True** or **False**.

- Examples:

```
>>> 2 < 3 -> True
```

```
>>> 3 < 1 -> False
```

```
>>> 4+5 > 2+3-> True
```

```
>>> 2 == 2 -> True
```

```
>>> 2 == 3 -> False
```

```
>>> 2 != 3 -> True
```

Comparison Operators

<

<

less than

<=

≤

less than or equal to

>

>

greater than

>=

≥

greater than or equal to

==

=

equal to

!=

≠

not equal to

Logical Operators

- Boolean expressions can be combined using logical operators.
 - and – returns true if both operands of the expressions are True
 - or – returns true if either of the operands are True.
 - not – returns the opposite of its operand
- `>>> 2 < 3 and 4 > 5 -> False`
- `>>> not (1 == 1) -> False`
- `>>> 2 < 3 or 4 > 5 -> True`

Operator Precedence

not – gets evaluated first and given the highest priority

and – gets evaluated second.

or – gets evaluated last

Example

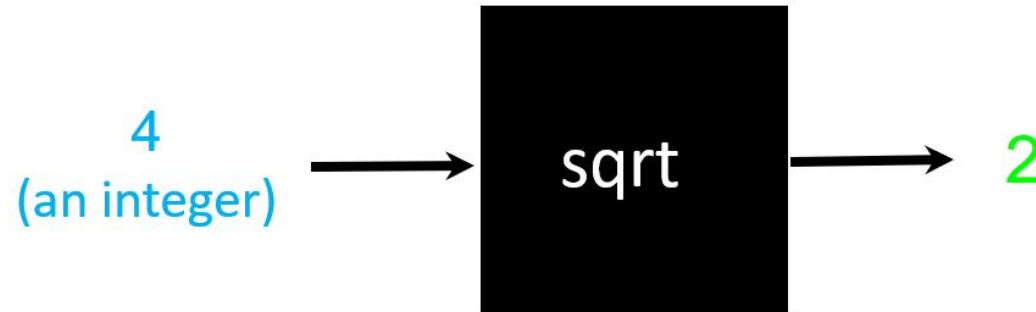
```
>>> not (1 == 0) or 2 == 1 and 1 == 1
```

```
>>> True or (2 == 1 and 1 == 1)
```

```
>>> True or False -> True
```

Functions

- Function is a named sequence of statements that performs a computation.
- A function is a block of code that performs a specific task.
- Functions are used to break down large programs into smaller, more manageable pieces. This makes code easier to read, understand, and maintain.



Function types

There are two kinds of functions in Python.

- Built-in functions that are provided as part of Python `type()`, `float()`, `int()`, `print()` ...
- User-defined functions that we define ourselves and then use

Built-in functions

```
>>> abs(-2) -> 2
```

```
>>> max(2,5,3) -> 5
```

```
>>> min(2,4,5) -> 2
```

Calling a function

The syntax to call a function is:

```
>>> function_name(argument)
```

- An argument is a value we pass into the function as its input when we call the function

```
>>> max(2,3,4)
```

2,3, and 4 are arguments.

Function Components

- Name of the function
- Parameters
- The sequence of statements that perform a computation

```
def function_name(parameter_1, parameter_2, ...):  
    # body of the function
```

Function Example

```
def add_numbers(number_1, number_2):  
    print(number_1 + number_2)
```

```
def multiply_numbers(number_1, number_2):  
    print(number_1 * number_2)
```

```
def get_area_of_triangle(base, height):  
    print ((base * height) / 2)
```

Calling functions

function_name(argument)

Example:

add_numbers(2,3)

multiply_numbers(2,3)

get_area_of_triangle(5,4)

Exercise

First convert the programs in the morning session by using functions to do the computation.

-> A function that prints hello and your name.

-> A function that computes area of rectangle.

-> A function that converts currency given the rate and amount.

Write a function that prints a square of a number

Write a function that prints a maximum of 4 numbers

Modules

- A module is a file that contains Python code.
- Modules can be used to organize code, to make code reusable, and to import code from other files.
- To use a module, you import it using the import statement.

```
>>> import math
```

```
>>> math.sqrt(4) -> 2.0
```

```
>>> math.log10(10) -> 1.0
```

Exercise

Write a function that calculates the distance between two points in the Cartesian coordinate system accepting four arguments: x coordinate of point 1(x1), y coordinate of point 1(y1), x coordinate of point 2(x2) and y coordinate of point 2(y2).

$$\text{distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Flow of Execution

- Execution always begins at the first statement of the program. Statements are run one at a time, in order from top to bottom.
- Function definitions do not alter the flow of execution of the program, but remember that statements inside the function don't run until the function is called.
- A function call is like a detour in the flow of execution. Instead of going to the next statement, the flow jumps to the body of the function, runs the statements there, and then comes back to pick up where it left off.

Demonstration

